

Tractable Problems in *AI Security* via Formal Methods

Forall R&D:

Guiding AI safety orgs through the formal methods explosion.

What formal methods can and can't claim

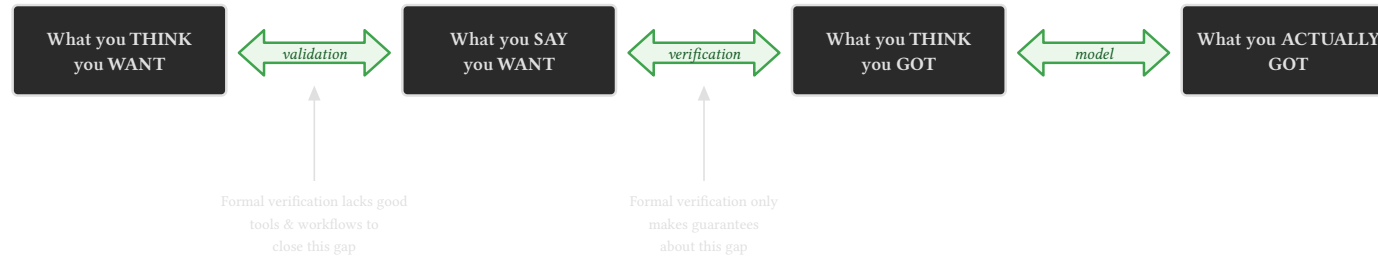
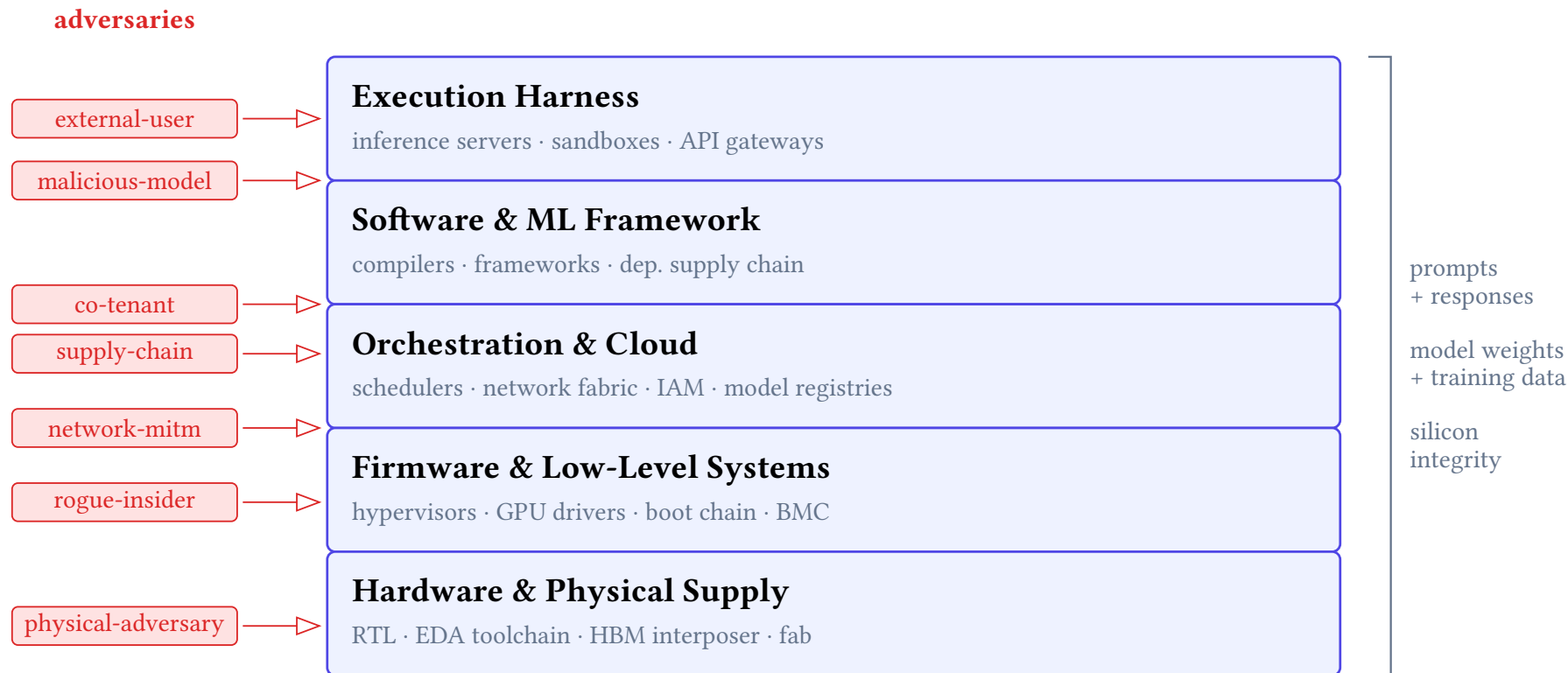


Figure 1: After Evan Miyazono. FM closes the middle gap only.

- **Elicitation gap** (left): is the spec the thing we actually wanted? Out of scope for the proof.
- **Verification gap** (middle): does the implementation meet the spec? This is what FM does.
- **Modeling gap** (right): does the proof's model of the hardware/OS/network match reality?

The ML training and inference stack



The tractable problems

17 entries on the website. Two flavors:

The tractable problems

17 entries on the website. Two flavors:

- **Enablers** unbottleneck a class of downstream work. Examples: adversarial robustness of FM tools, neuralese governance, spec elicitation.
- **Widgets** are concrete, scoped deliverables. Examples: seL4 GPU drivers, OCI runtime hardening, verified network tap.

The tractable problems

17 entries on the website. Two flavors:

- **Enablers** unbottleneck a class of downstream work. Examples: adversarial robustness of FM tools, neuralese governance, spec elicitation.
- **Widgets** are concrete, scoped deliverables. Examples: seL4 GPU drivers, OCI runtime hardening, verified network tap.

An enabler with many dependents should be addressed early. A widget contingent on an unsolved enabler should be sequenced accordingly.

Deep dive: three widgets, two enablers

I'll spend a slide or two on each.

1. OCI runtime hardening
2. seL4-native GPU drivers
3. Verified network tap
4. Spec elicitation and validation
5. Interpretability and governance of neuralese proof stacks

Widget 1: OCI runtime hardening

runc was never designed to resist an adversary *inside* the container. The CVE record reflects this.

Widget 1: OCI runtime hardening

runc was never designed to resist an adversary *inside* the container. The CVE record reflects this.

When the workload is an agent with tool use

Widget 1: OCI runtime hardening

writes code, executes it

Widget 1: OCI runtime hardening

“process isolation” is not a security boundary.

Widget 1: OCI runtime hardening

“process isolation” is not a security boundary.

Empirical work: BoxArena (Dougherty 2026) inverts the offensive framing. Fix the attacker model, vary the runtime. Five attack surfaces (fs, socket, process, network, syscall) across runc, runsc, crun, Kata.

OCI hardening: what the verified version looks like

Decomposes into tractable pieces:

OCI hardening: what the verified version looks like

Decomposes into tractable pieces:

- A formal model of the Linux ABI surface a hardened container actually uses (~80 syscalls, not 350+).
- A specification of the confinement policy (drop caps, read-only rootfs, seccomp allowlist, no-new-privs).
- A proof that the runtime's state machine preserves the policy across all reachable states.

OCI hardening: what the verified version looks like

Decomposes into tractable pieces:

- A formal model of the Linux ABI surface a hardened container actually uses (~80 syscalls, not 350+).
- A specification of the confinement policy (drop caps, read-only rootfs, seccomp allowlist, no-new-privs).
- A proof that the runtime's state machine preserves the policy across all reachable states.

Lineage: CLInc (Bevier 1989, gates to apps), seL4 (Klein 2009, microkernel functional correctness).

Widget 2: GPU drivers native to kernels other than linux

Multi-tenant GPU workloads run on hypervisors whose TCB is orders of magnitude too large to verify, with a proprietary ring-0 driver from the vendor.

Widget 2: GPU drivers native to kernels other than linux

Multi-tenant GPU workloads run on hypervisors whose TCB is orders of magnitude too large to verify, with a proprietary ring-0 driver from the vendor.

Candidates to host an ML-grade GPU stack:

- seL4 — most complete proof, no GPU support at all.
- CertiKOS — verified concurrent OS kernel with hypervisor extensions; layered refinement methodology, multicore.
- NOVA (not really a kernel) — partial proof covers concurrency and weak memory, host/guest split built in.

Widget 2: GPU drivers native to kernels other than linux

Multi-tenant GPU workloads run on hypervisors whose TCB is orders of magnitude too large to verify, with a proprietary ring-0 driver from the vendor.

Candidates to host an ML-grade GPU stack:

- seL4 — most complete proof, no GPU support at all.
- CertiKOS — verified concurrent OS kernel with hypervisor extensions; layered refinement methodology, multicore.
- NOVA (not really a kernel) — partial proof covers concurrency and weak memory, host/guest split built in.

The methodology (separation logic, abstract HW model) is settled. The blocker is that **the abstract hardware model does not exist**. No GPU vendor publishes a machine-checkable spec of their command processor.

seL4 GPU: the actual scope

Pick the smallest useful surface: command submission for one open stack (NVK/Nouveau or an AMDGPU subset).

seL4 GPU: the actual scope

Pick the smallest useful surface: command submission for one open stack (NVK/Nouveau or an AMDGPU subset).

Property: *no sequence of guest-supplied command packets causes the driver to program an IOMMU mapping or issue a DMA outside the guest's declared memory region.*

seL4 GPU: the actual scope

Pick the smallest useful surface: command submission for one open stack (NVK/Nouveau or an AMDGPU subset).

Property: *no sequence of guest-supplied command packets causes the driver to program an IOMMU mapping or issue a DMA outside the guest's declared memory region.*

Two stopping points:

- Verified module against a stubbed hardware model — shippable as reference.
- Same driver against a model co-developed with the vendor or with REMS (Sewell) — the research contribution.

Widget 3: Verified network tap

The threat: the SDN controller and fabric manager on a GPU cluster *are* the thing whose compromise we are worried about. Their own logs cannot be evidence.

Widget 3: Verified network tap

The threat: the SDN controller and fabric manager on a GPU cluster *are* the thing whose compromise we are worried about. Their own logs cannot be evidence.

The artifact: a passive (or active OEO) tap on the fiber, with an FPGA capture path proven against an LTL contract. North-south at the datacenter edge, east-west for storage, sampled for compute.

Widget 3: Verified network tap

The threat: the SDN controller and fabric manager on a GPU cluster *are* the thing whose compromise we are worried about. Their own logs cannot be evidence.

The artifact: a passive (or active OEO) tap on the fiber, with an FPGA capture path proven against an LTL contract. North-south at the datacenter edge, east-west for storage, sampled for compute.

Costs (Cankaya et al., 2026): under 0.01% of facility upfront for the edge tap; 0.3–0.75% for east-west. FPGA capture at 400 Gbps is a solved hardware problem (Arista 7130, Deutsche Börse).

Verified network tap: the LTL contract

$\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p))$	(P1) forwarding completeness
$\Box(\text{live_out}(p) \rightarrow \blacklozenge \text{live_in}(p))$	(P2) forwarding fidelity
$\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D'} \text{mon}(\text{image}(p)))$	(P3) mirror completeness
$\Box(\text{mon}(r) \rightarrow \blacklozenge(\exists p. \text{live_in}(p) \wedge \text{image}(p) = r))$	(P4) mirror soundness
$\Box(\text{live_out}(p) \rightarrow \neg \text{depends-on}(p, \text{mon-input}))$	(P5) no back-channel
$\Box(\text{fail} \rightarrow \Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p)))$	(P6) fail-to-wire

Verified network tap: the LTL contract

- | | |
|--|------------------------------|
| $\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p))$ | (P1) forwarding completeness |
| $\Box(\text{live_out}(p) \rightarrow \blacklozenge \text{live_in}(p))$ | (P2) forwarding fidelity |
| $\Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D'} \text{mon}(\text{image}(p)))$ | (P3) mirror completeness |
| $\Box(\text{mon}(r) \rightarrow \blacklozenge(\exists p. \text{live_in}(p) \wedge \text{image}(p) = r))$ | (P4) mirror soundness |
| $\Box(\text{live_out}(p) \rightarrow \neg \text{depends-on}(p, \text{mon-input}))$ | (P5) no back-channel |
| $\Box(\text{fail} \rightarrow \Box(\text{live_in}(p) \rightarrow \Diamond_{\leq D} \text{live_out}(p)))$ | (P6) fail-to-wire |

P5 is the noninterference property that defeats verifier collusion. P6 is fail-open: a fault must downgrade to a transparent pass-through, never a link cut.

Enabler 1: Spec elicitation and validation

Proofs are cheap. Specs are expensive. Every widget in this document terminates in a spec

Enabler 1: Spec elicitation and validation

a verified sampler, a hardened runtime, an IAM policy

Enabler 1: Spec elicitation and validation

and the spec is the part the proof oracle cannot write for you. Or rather, if it does, you have quietly handed over the thing you were trying to keep.

Enabler 1: Spec elicitation and validation

and the spec is the part the proof oracle cannot write for you. Or rather, if it does, you have quietly handed over the thing you were trying to keep.

Two subproblems:

- **Elicitation.** Stakeholder intent into a formal artifact. Requirements engineering with kernels, not PMs, as the consumer.
- **Validation.** The spec you wrote vs. the spec you meant. A proof of the wrong P launders a bug into a theorem.

Enabler 1: Spec elicitation and validation

and the spec is the part the proof oracle cannot write for you. Or rather, if it does, you have quietly handed over the thing you were trying to keep.

Two subproblems:

- **Elicitation.** Stakeholder intent into a formal artifact. Requirements engineering with kernels, not PMs, as the consumer.
- **Validation.** The spec you wrote vs. the spec you meant. A proof of the wrong P launders a bug into a theorem.

Handles: property-based testing (QuickCheck), spec mining (Ammons 2002), differential specification

Enabler 1: Spec elicitation and validation

two independently elicited specs, checked for agreement.

Enabler 2: Interp and governance of neuralese proof stacks

Warning: this would be a bad problem to have. We do not *condone* this problem.

The “proofs are cheap” world assumes a known, trusted, human-readable proof substrate: Lean, Rocq, Verus, Z3.

Enabler 2: Interp and governance of neuralese proof stacks

Warning: this would be a bad problem to have. We do not *condone* this problem.

The “proofs are cheap” world assumes a known, trusted, human-readable proof substrate: Lean, Rocq, Verus, Z3.

What if that assumption is violated? A proof oracle competent enough to make proofs free is also competent enough to write its own proof stack

Enabler 2: Interp and governance of neuralese proof stacks

one where human interpretability is not a first-class feature.

Enabler 2: Interp and governance of neuralese proof stacks

one where human interpretability is not a first-class feature.

Then spec elicitation runs against syntax nobody has ever seen, and proof checking runs in a kernel nobody has audited. The trusted base evaporates.

Enabler 2: Interp and governance of neuralese proof stacks

one where human interpretability is not a first-class feature.

Then spec elicitation runs against syntax nobody has ever seen, and proof checking runs in a kernel nobody has audited. The trusted base evaporates.

The enabler: keep the proof substrate inside what humans can read, audit, and govern. Otherwise every other widget in this document is built on opaque foundations.

What to do this weekend

If you came here to build, some hackathon-shaped ideas:

What to do this weekend

If you came here to build, some hackathon-shaped ideas:

- **Reproduce a BoxArena entry.** Pick a runtime, write the harness, run the five surfaces, publish the leaderboard row.
- **Spec a tiny piece of the OCI ABI in Lean or Rocq.** Even ten syscalls is useful.
- **Toy verified network tap in Cryptol or SymbiYosys.** P1, P2, P6 only. Stub the SerDes.
- **Try differential specification.** Pick a small library (parser, queue, token bucket). Elicit two independent specs from two collaborators. Diff them.
- **Sketch a proof-stack interp bench.** What would it take to audit a proof kernel you didn't design? Pick one tactic, write the inspector.

What to do over the next year

What to do over the next year

- If you're a **PL/FM researcher** looking for an AI-safety entry point: control protocols as a process calculus, or the verified OCI runtime.
- If you're an **ML person** worried about agentic deployment: BoxArena, sampler verification, context-window integrity.
- If you're **funding-shaped**: pay for the spec-elicitation work. It is the bottleneck the document points at.

What to do over the next year

- If you're a **PL/FM researcher** looking for an AI-safety entry point: control protocols as a process calculus, or the verified OCI runtime.
- If you're an **ML person** worried about agentic deployment: BoxArena, sampler verification, context-window integrity.
- If you're **funding-shaped**: pay for the spec-elicitation work. It is the bottleneck the document points at.

The website is the live menu: tractable.for-all.dev. The tags are layer × adversary × category. Filter to your shape.

What to do over the next year

- If you're a **PL/FM researcher** looking for an AI-safety entry point: control protocols as a process calculus, or the verified OCI runtime.
- If you're an **ML person** worried about agentic deployment: BoxArena, sampler verification, context-window integrity.
- If you're **funding-shaped**: pay for the spec-elicitation work. It is the bottleneck the document points at.

The website is the live menu: tractable.for-all.dev. The tags are layer × adversary × category. Filter to your shape.

Questions?

tractable.for-all.dev
quinn@for-all.dev